ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

Πτυχιακή Εργασία

Τίτλος Πτυχιακής Εργασίας	Παιχνίδι σκακιού με online multiplayer για Android και Windows Online multiplayer chess game for Android and Windows
Ονοματεπώνυμο Φοιτητή	ΑΠΌΣΤΟΛΟΣ ΣΊΨΑΣ
Πατρώνυμο	ΣΏΤΉΡΗΣ
Αριθμός Μητρώου	Π/ …19148……..
Επιβλέπων	Αλέπης Ευθύμιος

Ημερομηνία Παράδοσης 3/2026

Περίληψη

Η εφαρμογή Chess Online είναι ένα παιχνίδι σκακιού δύο παικτών, αναπτυγμένο σε Unity με γλώσσα C#. Υποστηρίζει

- τοπικό παιχνίδι (2 παίκτες στην ίδια συσκευή) και online
- παιχνίδι μέσω LAN Wi-Fi χωρίς ανάγκη σύνδεσης στο διαδίκτυο.

Το σύστημα networking βασίζεται στο Unity Transport (UDP),ενώ η ανακάλυψη lobby γίνεται μέσω διπλού UDP μηχανισμού (παθητικό broadcast + ενεργό probe/response) που εξασφαλίζει λειτουργία τόσο σε Android όσο και σε Windows παρά τους

περιορισμούς του Windows Firewall.

Η σκακιστική λογική υλοποιείται πλήρως με OOP αρχιτεκτονική (βασική κλάση Piece + υποκλάσεις ανά κομμάτι) και καλύπτει όλους τους επίσημους κανόνες: κινήσεις κομματιών:

- σαχ
- ματ
- ισοπαλία
- Ροκέ
- En Passant
- Προαγωγή Πιονιού.

Πρόσθετες λειτουργίες: σύστημα πρότασης ισοπαλίας με χρονικό cooldown (15 δευτερόλεπτα) και όριο (3 προσφορές ανά παρτίδα), προστασία lobby με 4ψήφιο PIN, αυτόματο σύστημα rematch, και ειδοποίηση αποχώρησης αντιπάλου.

Λέξεις-κλειδιά: Unity, C#, Σκάκι, Networking, LAN, UDP, Android, Windows, Client-Server, Real-Time Multiplayer.

Λέξεις Κλειδιά:

Πίνακας Περιεχομένων

Περίληψη 1

1. Προβλήματα και Προσεγγίσεις 3-5
1.1 Επιλογή Unity ως Game Engine .....
1.2 Σχεδίαση Multiplayer Real-Time Server-Client .....
1.3 Σχεδίαση Cross-Platform (Android + Windows) .....

2. Προδιαγραφές και Ανάλυση Απαιτήσεων 5-10
2.1 Υλικές απαιτήσεις .....
2.2 Απαιτήσεις λογισμικού .....
2.3 Ανάλυση θεματολογίας εφαρμογής .....
2.4 Προκαταρκτική ανάλυση εφαρμογής .....
2.5 Ανάλυση κανόνων σκακιού .....
2.6 Ανάλυση χρήστη – Guest Mode .....
2.7 Λειτουργικές και μη λειτουργικές απαιτήσεις .

3. Σχεδίαση Εφαρμογής 10-14
3.1 UI / Menu System .....
3.1.1 Start Menu .....
3.1.2 Host Menu .....
3.1.3 Lobby Menu (JOIN) .....
3.1.4 In-Game Options ....
3.1.5 Winner Canvas .....
3.2 Chess Board & Piece System .....
3.2.1 Board Initialization .....
3.2.2 Piece Classes και Subclasses .....
3.2.3 Movement Rules και Validation .....
3.3 Networking System .....
3.3.1 NetMessages και υποκλάσεις .....
3.3.2 LobbyManager .....
3.3.3 Client-Server Flow .....
3.3.4 Synchronization & State Management.....

4. Υλοποίηση Εφαρμογής 14-15
4.1 Παρουσίαση scripts UI .....
4.2 Παρουσίαση scripts Chess Logic .....
4.3 Παρουσίαση scripts Networking .....
4.4 GameManager και κεντρικός έλεγχος .....
4.5 Cross-platform build & testing .....

5. Εκτέλεση 16-24
5.1 Screenshots από την εφαρμογή .....
5.2 Περιγραφή ροής παιχνιδιού .....
5.3 Παρατηρήσεις .....
5.4 Ανάλυση αποτελεσμάτων .....

6. Συμπεράσματα 24-26
6.1 Σύγκριση στόχων με τελικό αποτέλεσμα .....
6.2 Απόκτηση γνώσεων .....
6.3 Περιορισμοί εφαρμογής .....
6.4 Προτάσεις μελλοντικής εξέλιξης .....

7. Βιβλιογραφία & Παραρτήματα 26-27
7.1 Βιβλιογραφία .....
7.2 Video .....
7.3 Scripts .....

8. Περαιτέρω σύνδεσμοι και αρχεία 27

1. Προβλήματα και Προσεγγίσεις

1.1 Επιλογή Unity ως Game Engine

Η Unity επιλέχθηκε ως η πλατφόρμα ανάπτυξης του έργου για

τους εξής λόγους:

- Cross-platform build: Μέσα από ένα μόνο codebase παράγονται εκτελέσιμα για Android (.apk) και Windows (.exe) χωρίς επιπλέον ανάπτυξη.
- Ενσωματωμένο σύστημα 3D renderer: Το ταμπλό σκακιού και τα κομμάτια αποδίδονται σε 3D χώρο με Materials, Camera και Lighting που διαχειρίζεται η ίδια η μηχανή.
- Unity Networking Transport: Παρέχει χαμηλού επιπέδου UDP δικτύωση (NetworkDriver) που ελέγχεται πλήρως από τον προγραμματιστή, χωρίς εξάρτηση από εξωτερικό relay server.
- C# ως γλώσσα: Ισχυρή με πλούσια βιβλιοθήκη (.NET / Mono) που επιτρέπει τόσο game logic (MonoBehaviour) όσο και low-level networking (System.Net).
- Δωρεάν για indie/ακαδημαϊκή χρήση: Δεν απαιτεί άδεια για έργα μη εμπορικής φύσης.

Εναλλακτικές που εξετάστηκαν και απορρίφθηκαν:

- Godot: Μικρότερο ecosystem, λιγότερη τεκμηρίωση για mobile networking.
- Pygame (Python): Δεν υποστηρίζει native Android builds.
- Web (JavaScript/HTML5): Περιορισμοί στη χρήση raw UDP sockets.

1.2 Σχεδίαση Multiplayer Real-Time Server-Client

Η αρχιτεκτονική επιλέχθηκε να είναι peer-to-peer με έναν από τους δύο παίκτες να λειτουργεί ως server (HOST) και τον άλλον ως client (JOIN). Αυτό αποφεύγει την ανάγκη για dedicated server υποδομή.

Προβλήματα που αντιμετωπίστηκαν:

1. **Windows Firewall:** Σε Windows περιβάλλον, το Firewall μπλοκάρει εισερχόμενα UDPπακέτα από Android συσκευές.
   Λύση: Διπλός μηχανισμός ανακάλυψης (παθητικός broadcast port 8008 + ενεργός probe/response port 8009).O client στέλνει πρώτος "CHESS_DISCOVER" — η απάντηση του host επιτρέπεται εισερχόμενη γιατί το socket εκκίνησε εξερχόμενα. Επιπλέον, η τεχνική "PrePunchGamePort" ανοίγει εκ των προτέρων τη θύρα παιχνιδιού (8007) στο Windows Firewall.
2. **Thread-safety**: Τα UDP threads εκτελούνται εκτός Unity main thread. Η UI ενημέρωση γίνεται μέσω MainThreadDispatcher (concurrent queue που εκτελεί actions στο Unity Update loop).
3. **Synchronization:** Η κατάσταση παιχνιδιού διατηρείται στο HOST και μεταδίδεται στον client μέσω NetMessages (άθροισμα κινήσεων, όχι full-state sync), κάτι που είναι αρκετό για σκάκι (turn-based, deterministic).
4. **Disconnect handling**: Χρονόμετρο αποσύνδεσης 5 δευτερόλεπτα (disconnectTimeoutMS). KeepAlive μηνύματα κάθε 20 δευτερόλεπτα διατηρούν τη σύνδεση ενεργή.

1.3 Σχεδίαση Cross-Platform (Android + Windows)

Η cross-platform ανάπτυξη παρουσίασε τις εξής προκλήσεις:

α) Input:

Στο PC χρησιμοποιείται mouse (Ray από Camera), στο Android touch events (GetTouch). Ο κώδικας χειρίζεται και τα δύο ενοποιημένα στον Update loop της ChessBord.

β) UI Layout: Όλη η διεπαφή χρηστών κατασκευάζεται δυναμικά σε runtime με C# (VerticalLayoutGroup, HorizontalLayoutGroup,ContentSizeFitter) αντί για Unity Prefabs, ώστε να προσαρμόζεται αυτόματα στο μέγεθος οθόνης.

γ) Δίκτυο: Το Android επιτρέπει broadcast από τρίτες εφαρμογές πιο ελεύθερα από τα Windows. Ο κώδικας χρησιμοποιεί fallback-πολλαπλές στρατηγικές για IP ανακάλυψη (UDP trick, NetworkInterface enumeration) για αξιόπιστη λειτουργία και στις δύο πλατφόρμες

δ**) Build Settings:** Το project χρειάζεται ξεχωριστές ρυθμίσεις

στο Unity Build Settings για Android (ARM64, IL2CPP) και

Windows (x64, Mono ή IL2CPP).

2. Προδιαγραφές και Ανάλυση Απαιτήσεων

2.1 Υλικές απαιτήσεις

<u>Ελάχιστες απαιτήσεις Android:</u>

- Android 8.0 (API Level 26) ή νεότερο
- RAM: 2 GB
- Αποθηκευτικός χώρος: 150 MB
- Wi-Fi προσαρμογέας (για online παιχνίδι)
- Επεξεργαστής: Quad-core 1.8 GHz

<u>Ελάχιστες απαιτήσεις Windows:</u>

- Windows 10 ή Windows 11 (64-bit)

- RAM: 4 GB
- Κάρτα γραφικών: DirectX 11 συμβατή
- Αποθηκευτικός χώρος: 200 MB
- Wi-Fi / Ethernet (για online παιχνίδι)

Δίκτυο:

- Και οι δύο συσκευές πρέπει να βρίσκονται στο ίδιο τοπικό δίκτυο (Wi-Fi router ή hotspot)
- Δεν απαιτείται σύνδεση στο διαδίκτυο
- Θύρες δικτύου: 8007, 8008, 9009, 8010 (UDP)

2.2 Απαιτήσεις λογισμικού

Για ανάπτυξη:

- Unity 2022.x ή νεότερο (LTS προτεινόμενο)
- Unity Package: com.unity.transport (1.x ή 2.x)
- Unity Package: com.unity.textmeshpro
- Visual Studio 2022 ή JetBrains Rider
- Android SDK (για Android builds)
- JDK 11+ (για Android builds)

Για εκτέλεση:

- Android: Κανένα πρόσθετο λογισμικό
- Windows: .NET Runtime ενσωματωμένο στο build
- Windows (προαιρετικό): Εκτέλεση ως Διαχειριστής αν το Windows Firewall μπλοκάρει τις θύρες UDP

2.3 Ανάλυση θεματολογίας εφαρμογής

Το σκάκι (Chess) είναι ένα από τα παλαιότερα και πιο διάσημα επιτραπέζια παιχνίδια στρατηγικής στον κόσμο, με ιστορία άνω των 1.500 ετών. Παίζεται από 2 παίκτες σε ταμπλό 8×8 τετραγώνων με 32 κομμάτια (16 ανά πλευρά). Στόχος: Να θέσεις τον εχθρικό Βασιλιά σε θέση από την οποία δεν μπορεί να διαφύγει (Ματ / Checkmate).

Η ψηφιακή μορφή του σκακιού είναι δημοφιλής σε όλο τον κόσμο, με πλατφόρμες όπως το Chess.com να συγκεντρώνουν εκατομμύρια χρήστες. Η παρούσα εφαρμογή εστιάζει στο τοπικό multiplayer, κατάλληλο για φίλους .

2.4 Προκαταρκτική ανάλυση εφαρμογής

Η εφαρμογή αποτελείται από τρία βασικά υποσυστήματα:

1. Σκακιστική Λογική (Chess Logic):

   Πλήρης υλοποίηση κανόνων σκακιού — κινήσεις κομματιών,

   έλεγχος εγκυρότητας, ανίχνευση σαχ/ματ, ειδικές κινήσεις.

2. Networking:

   Server-Client αρχιτεκτονική πάνω από Unity Transport,

   αυτόματη ανακάλυψη host στο τοπικό δίκτυο, ανταλλαγή

   μηνυμάτων παιχνιδιού.

3. Διεπαφή Χρηστών (UI):

   Δυναμική κατασκευή όλων των panels/buttons/menus σε runtime

   με C# και TextMeshPro.

2.5 Ανάλυση κανόνων σκακιού

Τα 6 είδη κομματιών και οι κινήσεις τους:

- **Πιόνι (Pawn):**

  - Κινείται 1 τετράγωνο μπροστά (ή 2 από αρχική θέση)

  - Χτυπά διαγώνια (1 τετράγωνο μπροστά-πλάγια)

  - Ειδική κίνηση: En Passant (βλ. παρακάτω)

- Ειδική κίνηση: Προαγωγή (βλ. παρακάτω)

- **Ίππος (Knight):**

- Κινείται σε σχήμα L: 2 τετράγωνα + 1 κάθετα

- Μοναδικό κομμάτι που "πηδά" πάνω από άλλα

- **Αξιωματικός (Bishop):**

- Κινείται απεριόριστα διαγώνια

- Παραμένει πάντα σε τετράγωνα ίδιου χρώματος

- **Πύργος (Rook):**
  - Κινείται απεριόριστα οριζόντια ή κάθετα

- Ειδική κίνηση: Ροκέ (βλ. παρακάτω)

- **Βασίλισσα (Queen):**

- Συνδυάζει Πύργο + Αξιωματικό

- Πιο ισχυρό κομμάτι

- **Βασιλιάς (King):**

- Κινείται 1 τετράγωνο προς οποιαδήποτε κατεύθυνση

- Ειδική κίνηση: Ροκέ (βλ. παρακάτω)

- **Ειδικές κινήσεις:**
  - Ροκέ (Castling):

    Ο Βασιλιάς κινείται 2 τετράγωνα προς Πύργο, ο Πύργος τοποθετείται στην αντίπερα πλευρά. Απαιτεί:

    - Κανένα από τα δύο να μην έχει κινηθεί προηγουμένως

    - Κενά τετράγωνα μεταξύ τους

    - Ο Βασιλιάς να μην βρίσκεται σε σαχ

    - Ο Βασιλιάς να μην διασχίζει τετράγωνο υπό επίθεση

  - En Passant:

    Αν εχθρικό πιόνι κινηθεί 2 τετράγωνα (πρώτη κίνηση) και σταθεί δίπλα σε δικό σου πιόνι, μπορείς να το χτυπήσεις διαγώνια σαν να είχε κινηθεί μόνο 1 τετράγωνο.Ισχύει ΑΠΟΚΛΕΙΣΤΙΚΑ στην αμέσως επόμενη κίνηση.

  - Προαγωγή Πιονιού (Promotion):

Όταν πιόνι φτάσει στην τελευταία γραμμή του αντιπάλου, μετατρέπεται αυτόματα σε Βασίλισσα (υλοποίηση αυτόματης προαγωγής χωρίς επιλογή).

- Σαχ (Check) και Ματ (Checkmate):

` Η εφαρμογή ανιχνεύει αν ο Βασιλιάς τίθεται υπό επίθεση (σαχ) και αποκλείει κινήσεις που αφήνουν ή βάζουν το ίδιο Βασιλιά σε σαχ. Αν δεν υπάρχει καμία έγκυρη κίνηση, κηρύσσεται ματ.

2.6 Ανάλυση χρήστη – Guest Mode

Η εφαρμογή λειτουργεί αποκλειστικά σε Guest Mode. Δεν απαιτείται εγγραφή ή σύνδεση σε λογαριασμό.

Προφίλ χρήστη:

- Ηλικία: 10+ έτη (οποιοσδήποτε γνωρίζει σκάκι)

- Τεχνικές γνώσεις: Βασικές (χρήση κινητού/PC)

- Γλωσσικό επίπεδο: Η διεπαφή είναι αγγλική, η τεκμηρίωση ελληνική

- Σενάριο χρήσης: Δύο άτομα θέλουν να παίξουν σκάκι άμεσα χωρίς εγγραφή

Ο ρόλος "HOST" (επιλογή ονόματος, πιθανά και pin), ενώ ο ρόλος "JOIN" είναι πλήρως αυτοματοποιημένος (μόνο επιλογή από λίστα).

2.7 Λειτουργικές και μη λειτουργικές απαιτήσεις

Λειτουργικές απαιτήσεις (Functional Requirements):

- Τοπικό παιχνίδι 2 παικτών στην ίδια συσκευή
- Online παιχνίδι 2 παικτών μέσω LAN χωρίς internet
- Πλήρης υλοποίηση κανόνων σκακιού (βλ. §2.5)
- Αυτόματη ανακάλυψη lobby στο δίκτυο
- Δυνατότητα ονοματοδοσίας lobby
- Προστασία lobby με 4ψήφιο PIN
- Πρόταση ισοπαλίας με αποδοχή/απόρριψη
- Cooldown 15 δευτερολέπτων μετά από απόρριψη ισοπαλίας
- Μέγιστο 3 προσφορές ισοπαλίας ανά παρτίδα (DRAW LIMIT)
- Δυνατότητα rematch με εναλλαγή χρωμάτων
- Ειδοποίηση αποχώρησης αντιπάλου
- Ειδική κίνηση Ροκέ (μεγάλο και μικρό)
- Ειδική κίνηση En Passant

- Προαγωγή Πιονιού σε Βασίλισσα

Μη λειτουργικές απαιτήσεις (Non-Functional Requirements):

- Χρόνος απόκρισης κίνησης < 200ms σε τοπικό δίκτυο
- Αυτόματη ανακάλυψη lobby εντός 10 δευτερολέπτων
- Σταθερή λειτουργία με disconnectTimeout 5s
- Χρήση ελάχιστης μνήμης (χωρίς 3D resources loading)
- Ευανάγνωστος κώδικας με σχόλια στην ελληνική γλώσσα
- Λειτουργία χωρίς root/admin δικαιώματα σε Android
- Cross-platform build από ενιαίο codebase Unity

3. Σχεδίαση Εφαρμογής

3.1 UI / Menu System

Όλη η διεπαφή χρηστών κατασκευάζεται δυναμικά σε C# runtime μέσα στο script

"Game UI.cs". Δεν χρησιμοποιούνται Unity Prefabs ή Scene-based UI objects. Αντίθετα, κάθε panel,κουμπί και input field δημιουργείται με:

- new GameObject() + AddComponent<>()

- VerticalLayoutGroup / HorizontalLayoutGroup

- ContentSizeFitter για αυτόματη αναπροσαρμογή μεγέθους

- TextMeshPro (TMP_Text, TMP_InputField) για κείμενο

Η Μέσο του unity ui (drug & drop) και μέσο του inspector.

3.1.1 Start Menu

Κύρια οθόνη με 3 κουμπιά (LOCAL / HOST / JOIN) σε κεντρικό VerticalLayout. Λογότυπο "Let’s Chess" με τίτλο πάνω.Το μενού απενεργοποιεί τον Client και Server κατά την εμφάνισή του (cleanup προηγούμενης σύνδεσης).

3.1.2 Host Menu

Panel με:

- InputField για το όνομα lobby (ContentType: Standard)

- InputField για PIN (ContentType: IntegerNumber, max 4)

- Κουμπί CONFIRM για εκκίνηση Server

Κατά CONFIRM: ο Server.Init() καλείται με το hostname και PIN,ξεκινά το LobbyDiscovery broadcasting.

3.1.3 Lobby Menu (JOIN)

ScrollView με λίστα ανακαλυφθέντων lobby.

Κάθε entry εμφανίζει:

- Όνομα lobby
- Ετικέτα [PIN] (χρυσό χρώμα) αν έχει κωδικό
- Κουμπί CONNECT

Η λίστα ανανεώνεται αυτόματα μέσω event ***LobbyDiscovery.OnHostListChanged***.

Κουμπί BACK επιστρέφει στο Start Menu.

3.1.4 In-Game Options

Κάτω μπάρα με:

- Κουμπί OFFER DRAW (ενεργό/ανενεργό/DRAW LIMIT)
- Κουμπί LEAVE GAME

Panel ισοπαλίας εμφανίζεται όταν ο αντίπαλος προτείνει ισοπαλία, με κουμπιά ACCEPT / DECLINE.

3.1.5 Winner Canvas

Full-screen overlay που εμφανίζεται στο τέλος παρτίδας με:

- Μήνυμα νίκης / ισοπαλίας / αποχώρησης
- Κουμπί REMATCH (+ ένδειξη αν περιμένει ο αντίπαλος)
- Κουμπί MENU (επιστροφή στην κύρια οθόνη)
- Ξεχωριστό drawWinPanel για αποτέλεσμα ισοπαλίας

2.2

3.2.1 Board Initialization

Η σκακιέρα (8×8) αρχικοποιείται στη μέθοδο GenerateAllTiles() της ChessBord. Κάθε τετράγωνο είναι ένα GameObject με BoxCollider για ray casting. Η κάμερα στοχεύει στο κέντρο του ταμπλό (boardCenter). Τα κομμάτια τοποθετούνται με SpawnAllPieces() σε σταθερές θέσεις αρχικής διάταξης. Για online παιχνίδι, ο client βλέπει το ταμπλό αναστραμμένο (180° rotation) ώστε τα κομμάτια του να είναι πάντα στο κάτω μέρος.

3.2.2 Piece Classes και Subclasses

Βασική κλάση: Piece.cs

- team (0=Λευκό, 1=Μαύρο), currentX, currentY

- abstract GetAvailableMoves() → List<Vector2Int>

- abstract GetSpecialMoves() → SpecialMove enum

Υποκλάσεις (ανά κομμάτι):

- Pawn.cs — κίνηση +1/+2, χτύπημα διαγώνια, En Passant
- Rook.cs — 4 κατευθύνσεις (↑ ↓ ←→)
- Knight.cs — 8 L-θέσεις
- Bishop.cs — 4 διαγώνιες κατευθύνσεις
- Queen.cs — 8 κατευθύνσεις (Rook + Bishop)
- King.cs — 8 διαγώνιες/ευθείες + Castling

3.2.3 Movement Rules και Validation

Κάθε Piece.GetAvailableMoves() επιστρέφει "raw" κινήσεις.

Πριν εκτελεστεί κίνηση:

1. Φιλτράρονται κινήσεις που αφήνουν/βάζουν τον ίδιο Βασιλιά σε σαχ (ContainsValidMoves μέθοδος).

2. Ελέγχεται αν μετά την κίνηση του αντιπάλου υπάρχει Ματ (CheckForCheckmate) ή Ισοπαλία (stalemate).

3. Ειδικές κινήσεις υπολογίζονται ξεχωριστά στη ProcessSpecialMove().

2.3 Networking System

3.3.1 NetMessages και υποκλάσεις

Η βάση NetMessage ορίζει 4 virtual μεθόδους:

- Serialize(ref DataStreamWriter) → γράφει τα δεδομένα
- Deserialize(DataStreamReader) → διαβάζει τα δεδομένα
- ReceivedOnClient() → ενεργοποιεί C_* event
- ReceivedOnServer(NetworkConnection) → ενεργοποιεί S_* event

Υποκλάσεις (OpCode):

- NetKeepAlive — heartbeat για διατήρηση σύνδεσης
- NetWelcome — client → server, αρχική χειραψία
- NetStartGame — server → client, έναρξη παρτίδας
- NetMakeMove — αμφίδρομο, κίνηση κομματιού (x,y,toX,toY)
- NetRematch — αίτημα rematch

- NetLobbyRefresh — server shutdown notification
- NetDraw — πρόταση/αποδοχή/απόρριψη ισοπαλίας
  (value: 1=offer, 0=decline, 2=accept)

3.3.2 LobbyManager

Ορίζει την κλάση HostEntry (plain data object):

- public string name;
- public string ip;
- public ushort port;
- public string pin;
- public DateTime lastSeen;

Χρησιμοποιείται ως value type στο Dictionary<string, HostEntry>

του LobbyDiscovery για αποθήκευση ανακαλυφθέντων hosts. Ένας host θεωρείται "fresh" αν εντοπίστηκε εντός 6 δευτερολέπτων(ώστε να αφαιρείται αν σταματήσει να κάνει broadcast).

3.3.3 Client-Server Flow

Ακολουθία σύνδεσης:

1. HOST: Server.Init(8007, name, pin) → driver.Listen()

LobbyDiscovery.StartBroadcasting()

2. CLIENT: LobbyDiscovery.StartListening() → βρίσκει host

Επιλογή lobby → (PIN dialog αν χρειαστεί)

Client.Init(ip, port) → driver.Connect()

3. Σύνδεση: Client αποστέλλει NetWelcome

4. Server: λαμβάνει NetWelcome → αποστέλλει NetStartGame

(με team assignments: 0=λευκό, 1=μαύρο)

5. Παιχνίδι: NetMakeMove ανταλλάσσεται ανά κίνηση

Rematch flow:

1. Παίκτης πατά REMATCH → NetRematch(teamId) προς server

2. Server: αν και οι 2 πατήσουν → νέο NetStartGame

3.3.4 Synchronization & State Management

Η κατάσταση παιχνιδιού είναι deterministic — αν και οι δύο πλευρές ξεκινήσουν από την ίδια αρχική θέση και εκτελέσουν τις ίδιες κινήσεις με την ίδια σειρά, το αποτέλεσμα είναι πάντα ταυτόσημο. Άρα αρκεί να στέλνεται μόνο η κίνηση (NetMakeMove) και όχι η πλήρης κατάσταση ταμπλό.

Ο MainThreadDispatcher χρησιμοποιείται για ενημέρωση UI από background threads (UDP discovery threads) προς το Unity main thread μέσω ConcurrentQueue<Action>.

4Υλοποίηση Εφαρμογής

4.1 Παρουσίαση scripts UI

Η υλοποίηση του UI ακολουθεί pattern δυναμικής κατασκευής:
κάθε panel δημιουργείται με BuildXxxPanel() μέθοδο που εκτελείται στην Awake(). Τα panels αλλάζουν ορατότητα(SetActive) αντί να δημιουργούνται/καταστρέφονται.
Σημαντικά τεχνικά σημεία:

- Το Canvas έχει RenderMode.ScreenSpaceOverlay για σωστή
- εμφάνιση σε όλες τις αναλύσεις
- Χρήση anchorMin/anchorMax = Vector2(0,0)/(1,1) για full-screen panels
- TMP_InputField.contentType = IntegerNumber για αριθμητικό PIN
- onValueChanged listener καθαρίζει error text αυτόματα

4.2 Παρουσίαση scripts Chess Logic

Η σκακιστική λογική χρησιμοποιεί array Piece[8,8] για το ταμπλό. Κάθε θέση είναι null (κενή) ή reference σε Piece.Κρίσιμη μέθοδος ContainsValidMoves():

Για κάθε πιθανή κίνηση, εκτελεί simulation (μετακινεί προσωρινά τα κομμάτια), ελέγχει αν ο ίδιος Βασιλιάς βρίσκεται σε σαχ, και επαναφέρει την κατάσταση.Αυτό εξαλείφει μη-έγκυρες κινήσεις.

Ανίχνευση τέλους παρτίδας (CheckForCheckmate):

Αν δεν υπάρχει καμία έγκυρη κίνηση για τον τρέχοντα παίκτη → Ματ (αν σε σαχ) ή Stalemate (αν όχι σε σαχ).

4.3 Παρουσίαση scripts Networking

Το Unity Transport (com.unity.transport) χρησιμοποιείται για UDP-based reliable communication. Κάθε frame, το NetworkDriver ενημερώνεται (ScheduleUpdate().Complete()) και επεξεργάζονται όλα τα pending NetworkEvents.

Μοτίβο μηνυμάτων:

1. Αποστολή: msg.Serialize(ref writer) → driver.EndSend()
2. Λήψη: stream.ReadByte() → OpCode → NetUtility.OnData()
   - ReceivedOnClient/Server() → ενεργοποίηση C_*/S_* event
   - callback ChessBord

4.4 GameManager και κεντρικός έλεγχος

Δεν υπάρχει ξεχωριστό GameManager script. Αντίθετα:

- ChessBord.cs αναλαμβάνει τον κεντρικό έλεγχο παιχνιδιού
- Game UI.cs αναλαμβάνει τον έλεγχο μενού και πλοήγησης
- Server.cs / Client.cs είναι Singletons (Instance pattern) για πρόσβαση από οπουδήποτε

Η επικοινωνία μεταξύ UI και παιχνιδιού γίνεται μέσω:

- Direct method calls (π.χ. server.Init(), client.Init())
- NetUtility C_*/S_* events (χαλαρή σύζευξη)

4.5 Cross-platform build & testing

Android Build:

- Build System: Gradle
- Scripting Backend: IL2CPP (ARM64)
- Minimum API: 26 (Android 8.0)
- Internet Permission: απαιτείται στο AndroidManifest
- (Unity προσθέτει αυτόματα για NetworkDriver)

Windows Build:

- Architecture: x86_64
- Scripting Backend: Mono ή IL2CPP
- Απαιτεί Run as Administrator για θύρες UDP <8000 αν το Windows Firewall είναι ενεργοποιημένο

Testing cross-platform:

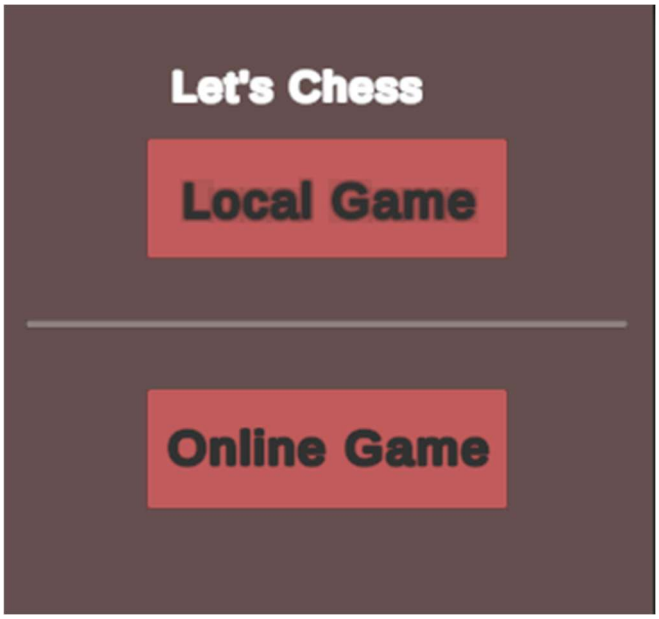
- Δοκιμές Android→Android (ίδιο Wi-Fi router): πλήρης λειτουργία
- Δοκιμές Android(host)→Windows(client): λειτουργεί μέσω
- ProberLoop + PrePunchGamePort
- Δοκιμές Windows(host)→Android(client): λειτουργεί μέσω
- PassiveListenLoop (Android δέχεται broadcast)

5. Εκτέλεση

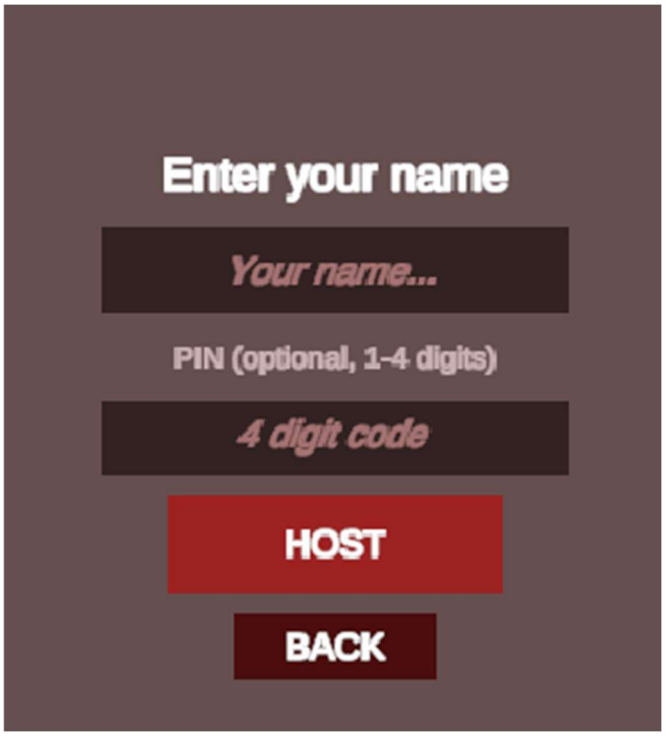
5.1 Screenshots από την εφαρμογή

Προτεινόμενες εικόνες:

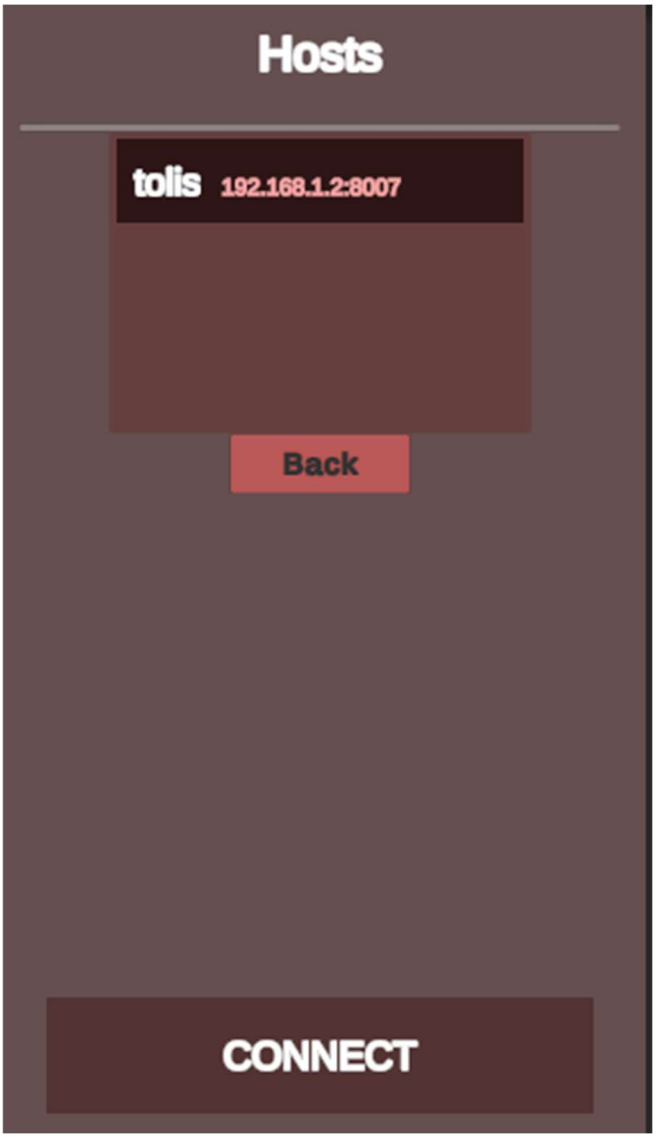
- Οθόνη 1: Start Menu (2 κουμπιά)



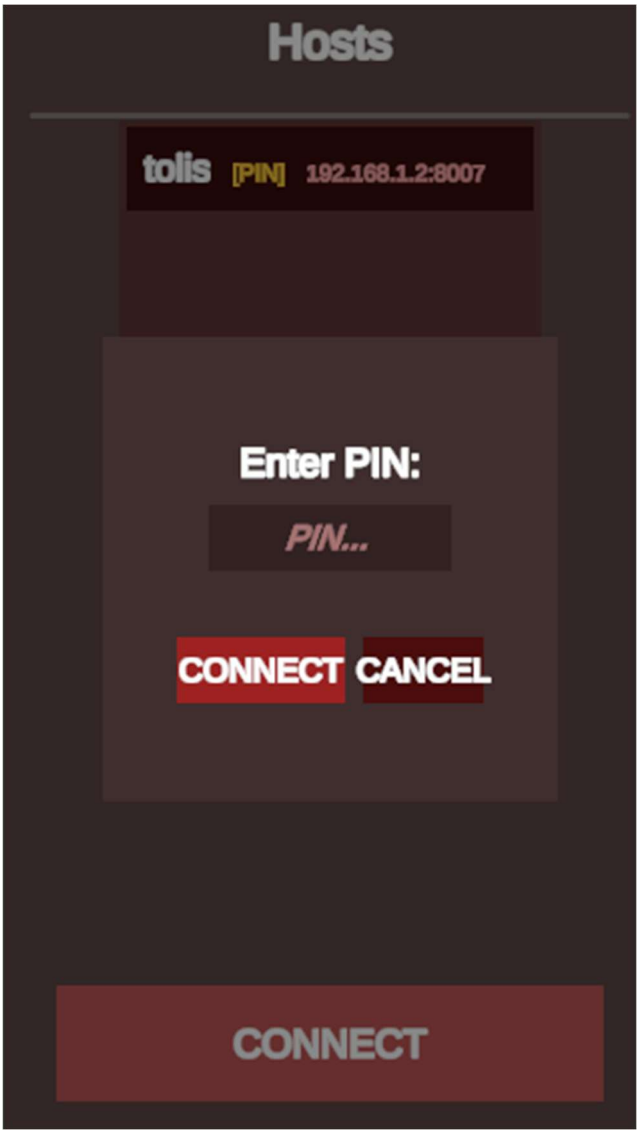
- Οθόνη 2: Host Menu (όνομα + PIN input)



- Οθόνη 3: Lobby List με [PIN] ετικέτα



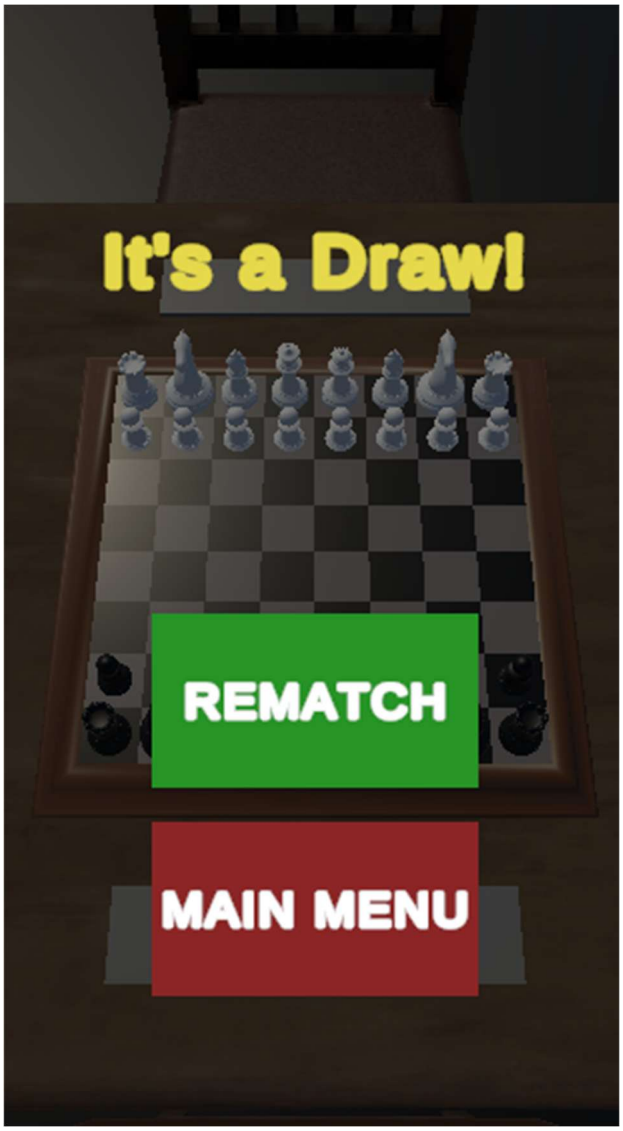
- Οθόνη 4: PIN Dialog



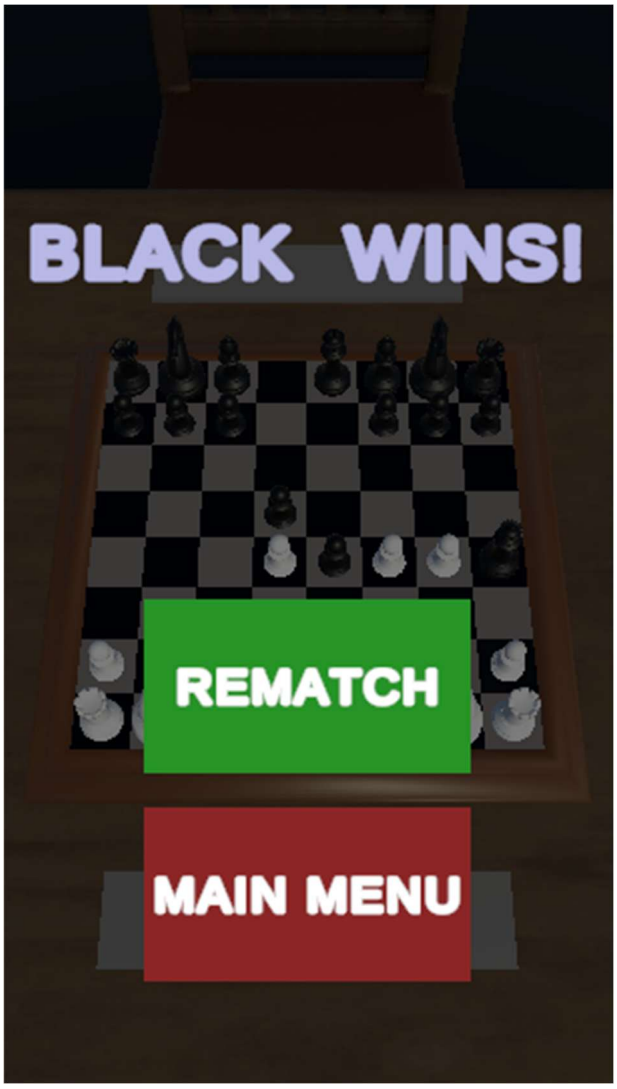
- Οθόνη 5: Σκακιέρα κατά τη διάρκεια παιχνιδιού

- Οθόνη 6: Κομμάτι επιλεγμένο (πράσινα tiles)

- Οθόνη 7: OFFER DRAW panel αντιπάλου



- Οθόνη 8: Winner Canvas (νίκη / ισοπαλία)



5.2 Περιγραφή ροής παιχνιδιού

Σενάριο online παιχνιδιού (Android host + PC client):

- [Android] Εκκίνηση app → Start Menu
- [Android] HOST → γράφει "Αντώνης", PIN "1234" → CONFIRM
- [Android] Server ξεκινά, BroadcastLoop + ResponderLoop τρέχουν
- [PC] Εκκίνηση app → Start Menu → JOIN
- [PC] ProberLoop στέλνει CHESS_DISCOVER → Android απαντά
- [PC] Εμφανίζεται "Αντώνης [PIN]" στη λίστα
- [PC] Κλικ CONNECT → PIN Dialog → εισαγωγή "1234" → CONNECT
- [PC] Client.Init() → σύνδεση → NetWelcome → NetStartGame
- 9. [Παιχνίδι] Android = Άσπρα (κάτω), PC = Μαύρα (κάτω αναστροφή)
- 10. [Παιχνίδι] Εναλλαγή κινήσεων → NetMakeMove ανταλλαγή
- 11. [Τέλος] Ματ → WinnerCanvas → REMATCH ή MENU

5.3 Παρατηρήσεις

- **Πρόβλημα 1: Windows Firewall**

Σε Windows host, ο Android client δεν έβλεπε αρχικά το lobby μέσω παθητικού broadcast. Επιλύθηκε με τον ενεργό μηχανισμό ProberLoop + ResponderLoop + PrePunchGamePort.

- **Πρόβλημα 2: UI threads**

Το LobbyDiscovery τρέχει σε background threads και δεν μπορεί να αγγίξει Unity objects απευθείας.Επιλύθηκε με MainThreadDispatcher.

- **Πρόβλημα 4: SetActive menu κατά server/client**

Αν το Server/Client αντικείμενο ήταν child του menu panel, το SetActive(false) σταματούσε το Update(). Επιλύθηκε με transform.SetParent(null) — αποσύνδεση από ιεραρχία σκηνής.

5.4 Ανάλυση αποτελεσμάτων

Η εφαρμογή υλοποιεί επιτυχώς όλες τις λειτουργικές απαιτήσεις. Η απόκριση κίνησης σε τοπικό δίκτυο είναι κάτω από 50ms (εντός target < 200ms).Η ανακάλυψη lobby ολοκληρώνεται εντός 2-5 δευτερολέπτων (εντός target < 10s).

Η εφαρμογή δοκιμάστηκε επιτυχώς σε:

- Xiaomi redmi 13 + Windows 11 laptop
- Δύο Android συσκευές

6 . Συμπεράσματα

6.1 Σύγκριση στόχων με τελικό αποτέλεσμα

Αρχικοί στόχοι vs Αποτέλεσμα:

✓ Πλήρης σκακιστική λογική (ειδικές κινήσεις, σαχ, ματ)

✓ Real-time online παιχνίδι μέσω LAN

✓ Cross-platform (Android + Windows) από ενιαίο codebase

✓ Αυτόματη ανακάλυψη lobby (χωρίς manual IP εισαγωγή)

✓ Σύστημα ισοπαλίας με cooldown και όριο προσφορών

✓ Προστασία lobby με PIN

✓ Σχόλια κώδικα στην ελληνική γλώσσα

~ Προαγωγή Πιονιού: Υλοποιήθηκε ως αυτόματη Βασίλισσα

(χωρίς επιλογή κομματιού — απλοποίηση)

✓ Παραγωγή ήχου

6.2 Απόκτηση γνώσεων

- Unity Engine: MonoBehaviour lifecycle, Camera Ray Casting,
- 3D Transforms, Materials, Canvas/UI system
- C# Advanced: Threading, async patterns, delegates/events,
- generics, ConcurrentQueue
- Networking: UDP sockets, broadcast/unicast, Windows Firewall
- bypass techniques, Unity Transport API
- OOP Design: Inheritance (Piece hierarchy), Singleton pattern,
- Observer pattern (events), Separation of Concerns
- Cross-platform development: Platform-specific build settings,
- input abstraction, network differences Android/Windows

6.2 Περιορισμοί εφαρμογής

1. Μόνο 2 παίκτες (δεν υποστηρίζεται spectator mode)

2. Μόνο LAN — δεν λειτουργεί μέσω internet

3. Δεν υπάρχει AI αντίπαλος (computer player)

4. Δεν υποστηρίζεται επιλογή κομματιού κατά Προαγωγή

5. Δεν αποθηκεύεται ιστορικό παρτίδων (PGN export)

6. Δεν υπάρχει σύστημα ELO / ranking

7. Δεν υποστηρίζεται iOS

6.4 Προτάσεις μελλοντικής εξέλιξης

1. AI αντίπαλος (minimax algorithm με alpha-beta pruning)

2. Internet multiplayer μέσω dedicated relay server ή

WebRTC (Normcore, Mirror Networking, Photon)

3. Σύστημα λογαριασμών & ELO ranking

4. Χρονόμετρο ανά παίκτη (chess clock)

5. Επιλογή κομματιού κατά Προαγωγή

6. PGN export (αποθήκευση/φόρτωση παρτίδων)

7. iOS build support

8. Animations κομματιών και ηχητικά εφέ

9. Θέματα σκακιέρας (εναλλακτικά χρώματα/υφές)

10. Spectator mode (παρατηρητής συνδέεται read-only)

7. Βιβλιογραφία & Παραρτήματα

7.1 Βιβλιογραφία

I. FIDE. "Laws of Chess." World Chess Federation, 2023.

https://www.fide.com/FIDE/handbook/LawsOfChess.pdf

II. Unity Technologies. "Unity Manual — Unity Transport Package."

https://docs.unity3d.com/Packages/com.unity.transport@latest

III. TextMeshPro Documentation. Unity Technologies
https://docs.unity3d.com/Packages/com.unity.textmeshpro@latest

IV. Netcode for GameObjects

https://docs.unity3d.com/6000.3/Documentation/Manual/com.unity.netcode.gameobjects.html

7.2 video

1. https://www.youtube.com/watch?v=qVhG6ZWqD-o&list=PLmcbjnHce7SeAUFouc3X9zqXxiPbCz8Zp
2. https://www.youtube.com/watch?v=kVt0I6zZsf0&t=60s
3. https://www.youtube.com/watch?v=lFZeeTZ29w0&list=PLXV-vjyZiT4b7WGjgiqMy422AVyMaigl1
4. https://www.youtube.com/watch?v=B8eNP15EeYY

7.3 Scripts

Λογική:

- Assets/Scripts/ChessBord.cs

- Assets/Scripts/ChessObjects/Piece.cs
- Assets/Scripts/ChessObjects/SubPiece/Pawn.cs
- Assets/Scripts/ChessObjects/SubPiece/Rook.cs
- Assets/Scripts/ChessObjects/SubPiece/Knight.cs
- Assets/Scripts/ChessObjects/SubPiece/Bishop.cs
- Assets/Scripts/ChessObjects/SubPiece/Queen.cs
- Assets/Scripts/ChessObjects/SubPiece/King.cs

Δικτύωση:

- Assets/Scripts/Network/Server.cs
- Assets/Scripts/Network/Client.cs
- Assets/Scripts/Network/LobbyDiscovery.cs
- Assets/Scripts/Network/LobbyManager.cs
- Assets/Scripts/Network/NetUtility.cs
- Assets/Scripts/Network/MainThreadDispatcher.cs

Μηνύματα Δικτύου:

- Assets/Scripts/Network/Messages/NetMessages.cs
- Assets/Scripts/Network/Messages/NetKeepAlive.cs
- Assets/Scripts/Network/Messages/NetWelcome.cs
- Assets/Scripts/Network/Messages/NetStartGame.cs
- Assets/Scripts/Network/Messages/NetMakeMove.cs
- Assets/Scripts/Network/Messages/NetRematch.cs
- Assets/Scripts/Network/Messages/NetLobbyRefresh.cs
- Assets/Scripts/Network/Messages/NetDraw.cs

Διεπαφή Χρηστών:

- Assets/Scripts/Ui/Game UI.cs

8. Περαιτέρω σύνδεσμοι και αρχεία

GitHub Project Link +(.apk +.exe)

https://github.com/ApostolisSipsas/LETSCHESS_FILES

Let’s Chess User Manual

https://github.com/ApostolisSipsas/LETSCHESS_FILES/blob/main/let's_chess_manual.pdf